FreEQ

How the three filters are built, how the cost was measured, and the code that runs.

FreEQ is a six-band equaliser written twice from the same specification. One realisation is Rust, on OpenPlugin 0.1.1-beta.1. The picture and the spectrum are GoodLookin DRI: the `goodlookindri` crate draws the face, and `goodlookindri-shell` opens the window. The other realisation is C++, compiled once with JUCE 8.0.17-security and once with JUCE 9.0.5. The published FreEQ source is the OpenPlugin crate. The JUCE figures below come from that comparison build. Each band is a peak, a high-pass, a low-pass, a low shelf, or a high shelf. The phase control selects minimum phase, linear phase, or Analog ZL. A +6 dB bell at 1 kHz measures 6.00 dB in the first two modes and 6.03 dB in Analog ZL, in both realisations. The long filters are played by overlap-save. In OpenPlugin that transform is BlockConvolver, planned with rustfft so the butterflies use Neon. The numbers below are from that build.

	OpenPlugin	JUCE 8.0.17	JUCE 9.0.5
Minimum, full patch	0.111%	0.229%	0.227%
Linear phase	0.160%	0.51%	0.50%
Analog ZL	0.159%	0.51%	0.50%
100 linear-phase EQs	16.7%	44%	43%

One hundred percent is one processor core for the length of the audio. Before the change, one linear-phase or Analog ZL equaliser used about 9%, and a hundred of them needed about nine cores.

The filters

A band is one second-order section. It remembers two input samples and two output samples, and mixes them with five coefficients taken from the Bristow-Johnson formulas. A peak lifts or cuts a neighbourhood of one frequency. A shelf tilts everything below a corner, or everything above it. A high-pass and a low-pass ignore the gain knob. Each is a single section, so the slope is 12 dB per octave. Frequency runs from 20 Hz to 20 kHz on a logarithmic ruler. Q sets the resonance.

Minimum phase runs the sections in series on every sample, then applies the output trim. No delay is inserted. The filters themselves still hold some frequencies back: a +6 dB bell at 1 kHz delays that region by about eleven samples, and frequencies far from the bell pass almost immediately. A peak whose gain is within a thousandth of a decibel of zero is left out of the sum, because that section is a straight wire. Shelves and passes are always included. The output meter records the loudest sample of the block and converts that one value to decibels.

Linear phase copies the magnitude of those six sections and throws their phase away. The magnitude is sampled at 1024 evenly spaced frequencies, the phase of every bin is set to zero, and an inverse transform produces an impulse that rings equally before and after its centre. Rotating that impulse places the centre on sample 512. Every frequency is then delayed by 512 samples, 10.7 ms at 48 kHz, which is the latency the plug-in reports. The ringing before the peak is the cost of a delay that does not depend on frequency.

Analog ZL is there because a biquad designed at the playback rate is a squeezed copy of an analogue filter. The squeeze is small at low frequencies and large near half the sample rate, so the top octave drifts off the analogue curve. The six sections are therefore designed as if the sample rate were sixteen times higher. Their magnitude is read back at the 1024 frequencies of the real playback rate and transformed into an impulse. The half that would have sounded before time zero is discarded, and the last 64 samples are faded out. The peak sits at the first sample, so the reported latency is zero. The filter is causal, so its delay still depends on frequency. What improved is the magnitude, which stays nearer the analogue shape through the top octave.

Both of those long filters are played by overlap-save, not by walking 1024 coefficients on every sample. Up to 1024 new samples are appended to a history of the input. The most recent 2048 samples are transformed, multiplied by the transform of the impulse, and transformed back. Only the newest samples of that result are kept; the older part is the overlap, and it is discarded. The impulse is padded with zeros to 2048 points before it is transformed, and that

spectrum is stored when the settings change. It is not rebuilt on every block. The spectrum drawn behind the yellow curve is a third transform, 4096 points with a Hann window, and it was switched off for the timings.

The two editors

Both windows are 980 by 640 points and keep that proportion when resized. The yellow line is the calculated response. The colour behind it is the spectrum display.



Figure 1. OpenPlugin. Minimum phase is selected.



Figure 2. JUCE. The 8.0.17 and 9.0.5 builds draw this same editor. Analog ZL is selected, and band 3 is a +6 dB bell at 1 kHz.

Measurement

Every run was 48 kHz, stereo, in blocks of 512 samples. The tone was a 1 kHz sine of amplitude 0.25, and its phase continued from block to block so the join did not click. The clock was the processor time charged to the program. After the equaliser had run, the same blocks were copied again with no filtering, and that copying time was subtracted. The result is the remaining time divided by the length of the audio. One hundred percent means the equaliser fills one core for as long as the audio lasts.

One equaliser was warmed for 187 blocks, about two seconds, and that warmup is not in the result. The clock then covered the next 750 blocks, exactly eight seconds. The pass was repeated six times. The first was discarded and the median of the other five was kept. For one hundred equalisers the audio was still eight seconds, with the same warmup, and the median was taken from three passes after one discarded pass. Each of the hundred had its own filter memory and received its own copy of the block. They were not chained. OpenPlugin finished, then the JUCE 8.0.17-security probe, then the JUCE 9.0.5 probe. The hundred-equaliser test followed that same order.

Idle sets all six bands to peaks at 0 dB. One band leaves that arrangement and sets band 3 to +6 dB at 1 kHz with Q equal to 1. Full makes band 1 a high-pass at 100 Hz, band 2 a low-pass at 8 kHz, and bands 3 to 6 peaks of +6 dB at Q equal to 2. The hundred-equaliser test used only the full setting. Trim was 0 dB. The spectrum display was off before the warmup. Both JUCE probes also checked the bell before the timing: linear phase read 6.00 dB at a latency of 512 samples, and Analog ZL read 6.03 dB at a latency of 0.

One equaliser

Table 1. Percent of realtime. Latency is samples at 48 kHz.

DSP	Load	OpenPlugin	JUCE 8.0.17	JUCE 9.0.5	Latency
Minimum	idle	0.012	0.025	0.021	0
Minimum	one band	0.070	0.104	0.080	0
Minimum	full	0.115	0.229	0.227	0
Linear	idle	1.12	0.51	0.48	512
Linear	one band	1.09	0.51	0.50	512
Linear	full	1.10	0.51	0.50	512
Analog ZL	idle	1.09	0.51	0.50	0
Analog ZL	one band	1.09	0.51	0.50	0
Analog ZL	full	1.09	0.51	0.50	0

One hundred equalisers

Table 2. One hundred independent copies of the full setting. One hundred percent is one core.

DSP	OpenPlugin	JUCE 8.0.17	JUCE 9.0.5	Latency each
Minimum	11.3	18.9	18.7	0
Linear	110	44.0	42.6	512
Analog ZL	110	44.1	42.8	0

Minimum phase follows the number of sections that are doing something. Idle is a few hundredths of a percent. The full patch, where all six run, is 0.115% in OpenPlugin and about 0.23% in JUCE. Linear phase and Analog ZL do not move when the patch changes, because both are the same 2048-point overlap-save of a 1024-tap impulse. JUCE spends about 0.50% on that transform and OpenPlugin about 1.10%, the difference being the speed of the transform itself. A hundred full minimum-phase equalisers take 11% of a core in OpenPlugin and about 19% in JUCE. A hundred of either FIR mode take 43 to 44% of a core in JUCE and about 110% in OpenPlugin. The same hundred previously needed about nine cores. JUCE 8.0.17-security and JUCE 9.0.5 agree. The choice between linear phase and Analog ZL remains a choice of delay and of top-octave magnitude, not a choice of cost.

The processing code

The two listings are the playback path. Minimum phase is the first branch. Linear phase and Analog ZL share the second. The impulse is built elsewhere, when a control moves, and only its spectrum is kept for the block loop. Line numbers are the lines in the files as they stand.

Listing 1. openplugin-eq/src/dsp.rs, the block path and the stored spectrum of the impulse.

```
        let trim = self.trim_lin;
        let mut active = [true; BANDS];
        for band in 0..BANDS {
            let kind = filter_kind(self.values[type_index(band)]);
            let gain = self.values[gain_index(band)];
            active[band] = !(matches!(kind, FilterType::Peaking) && gain.abs() < 1.0e-3);
        }
        let state = &mut self.channels[channel];
        let mut peak = 0.0f32;
        for sample in samples.iter_mut() {
            let mut y = *sample;
            for (band, on) in state.bands.iter_mut().zip(active) {
                if on {
                    y = band.process_sample(y);
                }
            }
            y *= trim;
            peak = peak.max(y.abs());
            *sample = y;
        }
        self.push_meter(channel, peak);
    }

    fn process_fir(&mut self, channel: usize, samples: &mut [f32]) {
        self.convolver.process(channel, samples);
        let trim = self.trim_lin;
        let mut peak = 0.0f32;
        for sample in samples.iter_mut() {
            *sample *= trim;
            peak = peak.max(sample.abs());
        }
        self.push_meter(channel, peak);
    }

    fn push_meter(&mut self, channel: usize, peak: f32) {
        if channel >= 2 {
            return;
        }
        let db = if peak < 1.0e-8 { -80.0 } else { 20.0 * peak.log10() };
        self.meter[channel] = if db > self.meter[channel] {
            db
        } else {
            self.meter[channel] * 0.9995 + db * 0.0005
        };
    }

    fn prepare_kernel(&mut self) {
        let channels = self.channels.len().max(1);
        if self.convolver.channels() != channels {
            self.convolver = BlockConvolver::new(channels);
        }
        self.convolver.set_kernel(&self.fir);
    }

    fn apply_coeffs(&mut self) {
        let sr = self.sample_rate;
        self.phase_mode = (self.values[PHASE_INDEX].round() as i32).clamp(0, 2) as u8;
        self.trim_lin = 10.0_f32.powf(self.values[TRIM_INDEX] / 20.0);
        for channel in &mut self.channels {
            configure_channel(channel, &self.values, sr);
        }
        let mut prototype = Channel::new();
        configure_channel(&mut prototype, &self.values, sr);
        self.fir = match self.phase_mode {
            1 => design_linear(&prototype),
            2 => design_analog_zl(&self.values, sr),
            _ => identity_fir(),
        };
        self.prepare_kernel();
    }
}

fn configure_channel(channel: &mut Channel, values: &[f32; PARAM_COUNT], sr: f32) {
    for band in 0..BANDS {
        let kind = filter_kind(values[type_index(band)]);
        let freq = values[freq_index(band)];
        let q = values[q_index(band)].max(0.2);
        let gain = if matches!(kind, FilterType::HighPass | FilterType::LowPass) {
            0.0
        } else {
            values[gain_index(band)]
        };
```

```
        channel.bands[band].configure(kind, sr, freq, q, gain);
    }
}

fn filter_kind(type_value: f32) -> FilterType {
    match type_value.round() as i32 {
        1 => FilterType::HighPass,
        2 => FilterType::LowPass,
        3 => FilterType::LowShelf,
        4 => FilterType::HighShelf,
        _ => FilterType::Peaking,
    }
}

fn host_stages(values: &[f32; PARAM_COUNT], rate: f32) -> Vec<Stage> {
    let mut stages = Vec::with_capacity(BANDS);
    for band in 0..BANDS {
        stages.push(stage_for_band(values, band, rate));
    }
    stages
}

fn stage_for_band(values: &[f32; PARAM_COUNT], band: usize, rate: f32) -> Stage {
    let kind = filter_kind(values[type_index(band)]);
    let gain = if matches!(kind, FilterType::HighPass | FilterType::LowPass) {
        0.0
    } else {
        values[gain_index(band)]
    };
    bilinear_stage(kind, rate, values[freq_index(band)], values[q_index(band)].max(0.2), gain)
}

fn design_linear(proto: &Channel) -> [f32; FIR_LEN] {
    let mut scratch = proto.clone();
    scratch.reset();
    let mut ir = [0.0f32; FIR_LEN];
    ir[0] = run_bands(&mut scratch, 1.0);
    for sample in ir.iter_mut().skip(1) {
```

Listing 2. juce-eq/Source/EqFilters.cpp, the same path. The transform is juce::dsp::FFT of 2048 points.

```
float EqFilters::processSample (int channel, float x)
{
    processChannel (channel, &x, 1);
    return x;
}

void EqFilters::processChannel (int channel, float* samples, int numSamples)
{
    if (samples == nullptr || numSamples <= 0 || channel < 0 || channel >= (int) channels.size())
        return;
    if (phaseMode != 0)
    {
        int offset = 0;
        while (offset < numSamples)
        {
            const int count = juce::jmin (1024, numSamples - offset);
            firChunk (channel, samples + offset, count);
            offset += count;
        }
        return;
    }
    bool active[eqparams::bands];
    for (int band = 0; band < eqparams::bands; ++band)
        active[band] = bandIsRunning (band);
    auto& state = channels[(size_t) channel];
    float peak = 0.0f;
    for (int i = 0; i < numSamples; ++i)
    {
        float y = samples[i];
        for (int band = 0; band < eqparams::bands; ++band)
            if (active[band])
                y = state.bands[band].processSample (y);
        y *= trimLin;
        peak = juce::jmax (peak, std::abs (y));
        samples[i] = y;
    }
    pushMeter (channel, peak);
}

bool EqFilters::bandIsRunning (int band) const
{
    const int type = kind (values[eqparams::typeIndex (band)]);
    return ! (type == 0 && std::abs (values[eqparams::gainIndex (band)]) < 1.0e-3f);
}

void EqFilters::pushMeter (int channel, float peak)
{
    if (channel < 0 || channel >= 2)
        return;
    const float db = peak < 1.0e-8f ? -80.0f : 20.0f * std::log10 (peak);
    meter[channel] = db > meter[channel] ? db : meter[channel] * 0.9995f + db * 0.0005f;
}

void EqFilters::prepareKernel()
{
    if ((int) convWork.size() < convFft * 2)
        convWork.resize ((size_t) convFft * 2);
    std::fill (convWork.begin(), convWork.end(), 0.0f);
    for (int i = 0; i < firLen; ++i)
        convWork[(size_t) i] = fir[i];
    fft2048.performRealOnlyForwardTransform (convWork.data(), false);
    std::copy (convWork.begin(), convWork.begin() + convFft * 2, kernel);
}

void EqFilters::firChunk (int channel, float* samples, int count)
{
    auto& state = channels[(size_t) channel];
    for (int i = 0; i < count; ++i)
    {
        state.convHist[state.convPos] = samples[i];
        state.convPos = (state.convPos + 1) % convFft;
        if (state.convWritten < 0xffffffffffffffffULL)
            ++state.convWritten;
    }
    if ((int) convWork.size() < convFft * 2)
        convWork.resize ((size_t) convFft * 2);
    std::fill (convWork.begin(), convWork.end(), 0.0f);
    for (int index = 0; index < convFft; ++index)
    {
        const int age = convFft - 1 - index;
        if ((juce::uint64) age < state.convWritten)
        {
            const int slot = (state.convPos + convFft - 1 - age) % convFft;
            convWork[(size_t) index] = state.convHist[slot];
        }
    }
    fft2048.performRealOnlyForwardTransform (convWork.data(), false);
    for (int index = 0; index < convFft; ++index)
    {
        const float xr = convWork[(size_t) index * 2];
        const float xi = convWork[(size_t) index * 2 + 1];
```

```
        const float hr = kernel[(size_t) index * 2];
        const float hi = kernel[(size_t) index * 2 + 1];
        convWork[(size_t) index * 2] = xr * hr - xi * hi;
        convWork[(size_t) index * 2 + 1] = xr * hi + xi * hr;
    }
    fft2048.performRealOnlyInverseTransform (convWork.data());
    const int start = convFft - count;
    float peak = 0.0f;
    for (int i = 0; i < count; ++i)
    {
        const float y = convWork[(size_t) (start + i)] * trimLin;
        peak = juce::jmax (peak, std::abs (y));
        samples[i] = y;
    }
    pushMeter (channel, peak);
}
```